Infosys®

POWERED BY INTELLECT
DRIVEN BY VALUES

RDBMS - Day6

Education and Research

Agenda

- Motivation
- Transaction
- Types of Transaction Systems
- Transaction Properties
- Requirements for an OLTP system
  - Integrity
  - Concurrency
- Locking
- Granularity of Locking
- Intent Locking
- Deadlock

Motivation

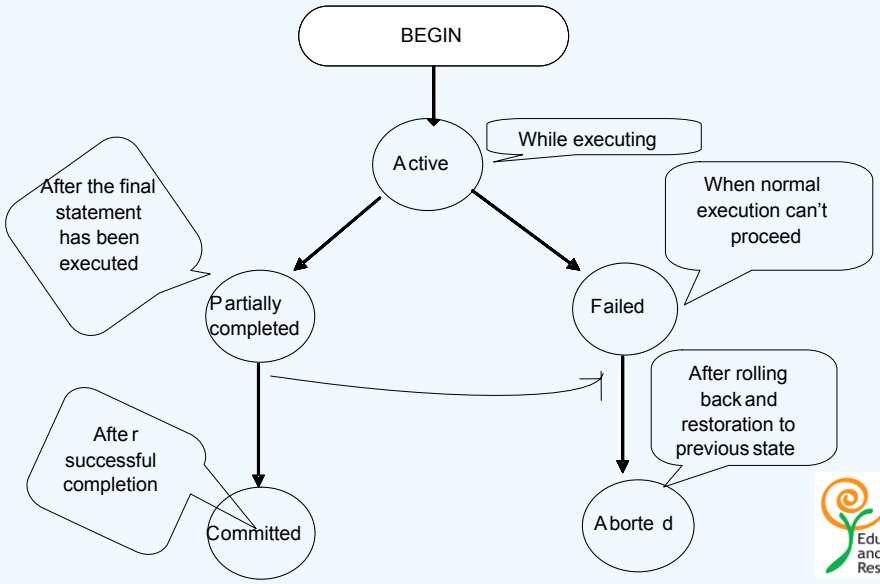
The biggest responsibility of the modern day information system is

- To simulate the manual system,
- To record every transaction that the organization undertakes,
- Capture the day-to-day activities in the life cycle of an enterprise and
- Help the organization to make quick and correct decisions based on the data.
- Protect the data from unauthorized access.
- Recovering the data in case of failures.

Transaction

Logical unit of program execution that takes a database from one consistent state to another **consistent state**

State diagram of a Transaction



Transaction Systems

- Batch Transaction Processing System
- On-Line Transaction Processing System
- Real time Transaction Processing System

Transaction Properties

- Atomicity
- Consistency
- Isolation
- Durability

Copyright © 2004, Infosys Technologies Ltd

ER/CORP/CRS/DB07/003
Version No: 2.0

Atomicity- Transaction management

•DBMS keeps track of the old value of a data on which the transaction acts so that, if there is a failure, the old value of the data is restored as though no transaction acted on it

`

Consistency-Application programmer

•To ensure that the database remains consistent after execution of the transaction

Isolation-Concurrency control

• To ensure concurrent execution of transactions results in a state equivalent to that obtained by sequential execution

Durability-Recovery management

•To ensure that after successful transaction completion, all updates persist irrespective of system failures

Requirements for an OLTP system

- Integrity
- Concurrency

Infosys®

POWERED BY INTELLECT
DRIVEN BY VALUES

Integrity

Education and Research

Integrity

- Refers to :
  - Correctness and completeness of data
  - Validity of individual items
  - Preservation of interrelationships in the DB
- Data integrity constraints:
  - Required data
  - Domain integrity
  - Entity integrity
  - Referential integrity

Copyright © 2004, Infosys Technologies Ltd

To preserve correctness of data, any RDBMS imposes data integrity constraints. These constraints restrict the data values that can be inserted / modified. The constraints commonly found in RDBMSs are as found in the slide.

We will discuss in detail about these constraints in the following slides..

Required data

- Requires a column to contain Non-NULL values
- Indicated with the key word **NOT NULL** in create statement

An example:

```
CREATE TABLE EMPLOYEE (

    EMP_NO        integer  NOT NULL,
    EMP_NAME      varchar (15) ,
    EMP_AGE       integer,
.......
............
```

Copyright © 2004,
Infosys Technologies Ltd

An insert into the table without a value for the column with NOT NULL constraint fails

An update trying to set the value of the column to a NULL value fails

Domain Integrity-Check constraint

- Specify a range of values that a column can take.
- Used to specify business rules.
- Specified in the create statement

An example:

```
CREATE TABLE EMPLOYEE (
        EMP_NO          integer  NOT NULL,
        ……————
        …….
        EMP_LOC         varchar(15)
        CHECK ( EMP_LOC in ( 'BANGALORE',
        'BOMBAY','DELHI')));
```

Copyright © 2004, Infosys Technologies Ltd

Check constraint is like a search condition. This will produce a true/false value. The condition specified in the check constraint is verified by the RDBMS for every insert and update operations. If any violation is observed, the operation fails.

Entity Integrity

- The database models the outside world
- A table's primary key should have unique values so that it represents some real world entity
- The requirement that the primary key be unique is called "Entity integrity constraint"

Copyright © 2004, Infosys Technologies Ltd

ER/CORP/CRS/DB07/003
Version No: 2.0

When a primary key is defined on a table, DBMS automatically checks to see if any duplicates are entered, whenever an insert or update is attempted on the table. If any duplicate is found , the operation fails with an error message.

Referential Integrity

- Ensures that the integrity of the parent-child relationship between tables created by primary and foreign keys is preserved

- Issues:
  - Inserting a new child row
  - Updating foreign key in a child row
  - Updating the primary key in a parent row
  - Deleting a parent row

Copyright © 2004, Infosys Technologies Ltd

Let us take the first issue: inserting a new child row:

When we try to insert a row in the child table, its foreign key value must be the same as one of the primary key values in a parent table. For example, The deptno is a foreign key in the employee table which refers to the dept# in the department table. let us say, we try to insert a row in the employee table, with a department number which is not there in the department table, then it indicates the data is wrong. So , such a insert should not be allowed. Databases take care of this situation automatically

A similar check is performed when we try to modify the foreign key field

The last two issues are similar:
Updating /deleting the parent row. Let us say, we are dissolving a department altogether, what will happen to the eployees who belong to the department?

We have to do one of the following:
1. Prevent the dept being deleted until all its employees are re allotted to some other dept
2. Automatically delete the employees who belong to the dept
3. Set the dept column for the employees who belong to the dept to NULL
4. Set the dept column of these employees to some default value which indicates they are currently not allotted to any dept

To achieve this, 4 options can be set in CREATE table command:
ON DELETE RESTRICT
ON DELETE SET NULL
ON DELETE SET DEFAULT
ON DELETE SET CASCADE

For ex:
```
CREATE table ORDERS
        ( Order_Num Integer NOT NULL,
        ……————————————
        ……————————————
Foreign key (Cust)  REFERENCES  Customers
        ON DELETE  CASCADE,
Foreign key (Rep)  REFERENCES  SalesReps
        ON DELETE  SET NULL,
        ON UPDATE  CASCADE,
Foreign key (Mfr, Product)
        REFERENCES Products
        ON DELETE  RESTRICT
```

Infosys®

POWERED BY INTELLECT
DRIVEN BY VALUES

Concurrency

Education and Research

Concurrency

- Two or more users access a database concurrently
- Problems associated with concurrent execution:
  - Lost update
  - Dirty read
  - Non repeatable read
  - Phantom records
- Concurrency techniques:
  - Locking
  - Time stamping

Lost update

Time	Hilary's Deposit	Balance	Kevin's Deposit
10:22	Read Balance (1500)	1500	
10:23	Balance=1500+2000		
10:24			Read Balance (1500)
10:25	Write new Balance (3500)	3500	
10:26	Commit		
10:27			Balance=*1500*+1800
10:28		3300	Write new Balance (3300)
10:29			Commit

Dirty Read

Time	Hilary's Deposit	Balance	Kevin's Deposit
11:22	Read Balance (1500)	1500	
11:23	Balance=1500+2000		
11:24	Write new Balance (3500)	3500	
11:25			Read Balance *(3500)*
11:26	Rollback		
11:27			Balance= *3500* + 1800
11:28		5300	Write new Balance *(5300)*
11:29			Commit

Copyright © 2004, Infosys Technologies Ltd

Incorrect summary

Time	Hilary's Transfer	Balance	Summary Transaction
12:22	Read Hilary's Balance (1500)	1500	Sum = 0
12:23	Balance=1500-500		Read Hilary's Balance *(1500)*
12:24	Write new Balance (1000)	1000	
12:25			Sum=Sum + Balance *(1500)*
12:26	Read Evelyn's Balance (1500)	1500	
12:27	Balance=1500 + 500		
12:28	Write new Balance (2000)	2000	
12:29	Commit	2000	
12:30			Read Evelyn's Balance *(2000)*
12:31			Sum=Sum + Balance *(3500)*
12:32		3500	Write Sum *(3500)*
12:33			Commit

Phantom Record

Time	Create account		Total Accounts
13:22			
13:23			
13:24			Read the total number of accounts in the bank as total
13:25	Create account for Simon with a deposit of 500		
13:26	Create account for Mike with a deposit of 1000		
13:27			
13:28	Commit		
13:29			Write Total
13:30			Commit

Copyright © 2004, Infosys Technologies Ltd

Solution to the Problems – Serialization…

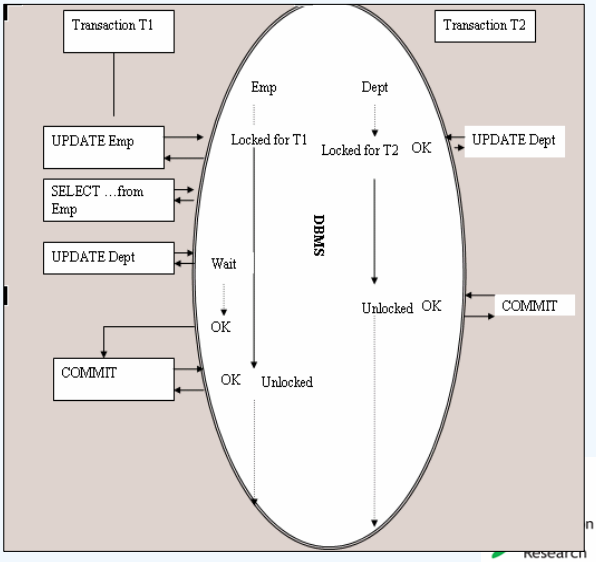
- To make every transaction follow each other.
- Achieved by setting following rules on transactions:
  - If any row is being modified, then do not allow any other transaction either to read or write that row until the first transaction completes.
  - If a transaction is reading a particular row, prevent other transactions from making any changes to that row until the first transaction completes.
  - If a transaction is reading some data, do not allow any other transaction to insert new rows into the same table until the first transaction completes. This will avoid problems like phantom records.

Solution to the Problems – Serialization.

- **Serialization** can be achieved using:
  - Locking
  - Time stamping

Locking

- A lock is a variable associated with each data item in a database.
- When updated by a transaction, DBMS locks the data item
- serializability could be maintained by this.
- Lock could be Shared or Exclusive



- An example ->

Granularity of locks

- The granular level at which a resource can be locked
- A database consists of several items that form a hierarchy. For example, the general hierarchy is:
  - Database
  - Tablespace
  - Table
  - Row

Intent Locking

- Only the intention of locking is expressed at the ancestor node of the required resource.
- Also called Parent Child locking.
- Increase the concurrency.

Types of Intent Locking

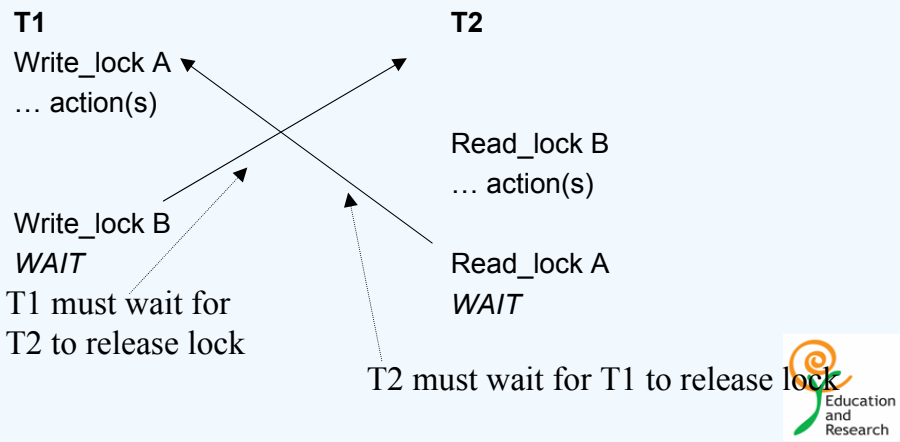
- IS (Intent Share)
- IX (Intent Exclusive)
- SIX (Shared Intent Exclusive)

Lock Compatibility matrix

	S	X	IS	IX	SIX
S	Y	N	Y	N	N
X	N	N	N	N	N
IS	Y	N	Y	Y	Y
IX	N	N	Y	Y	N
SIX	N	N	Y	N	N

Deadlock

- Occurs when two or more separate processes compete for resources held by one another.



Copyright © 2004, Infosys Technologies Ltd

•Deadlock occurs when one transaction is waiting on another to release a lock it needs, and vice versa - each then will wait forever for the other

•If a deadlock occurs one of the offending transactions must be *rolled back* to allow the other to proceed

•There are various methods for choosing which transaction to roll back when a deadlock is detected

- •Time (how long the transactions have been running)
- •Data updated
- •Data remaining to update

•There are schemes for preventing deadlock. But, most DBMSs allow them to occur and resolve when they are detected

Detection may be based on:

- •Timeout
- •Wait-for-graph (shows which transactions are waiting on which other transactions for a lock)

Deadlock- Example

Time	Transaction BalanceUpdate	Transaction LoanUpdate
10:22	Lock ACC_DETAILS	Lock LOAN_DETAILS
10:23	Update ACC_DETAILS	Update LOAN_DETAILS
10:24	Try for lock on LOAN_DETAILS	Try for lock on ACC_DETAILS
10:25	Wait for lock	Wait for lock
10:26	Wait for lock	Wait for lock
10:27	Wait for lock	Wait for lock
10:28	Wait for lock	Wait for lock

Summary

- Transaction is a logical unit of work which takes the database from one consistent state to the other
- Atomicity, consistency, isolation and durability are the ACID properties of a transaction
- Data integrity and Security are enforced using SQL DDL statements
- Transactions should be able to concurrently execute without affecting the consistency of the database
- Locking is a mechanism of achieving such controlled concurrency

Thank You!